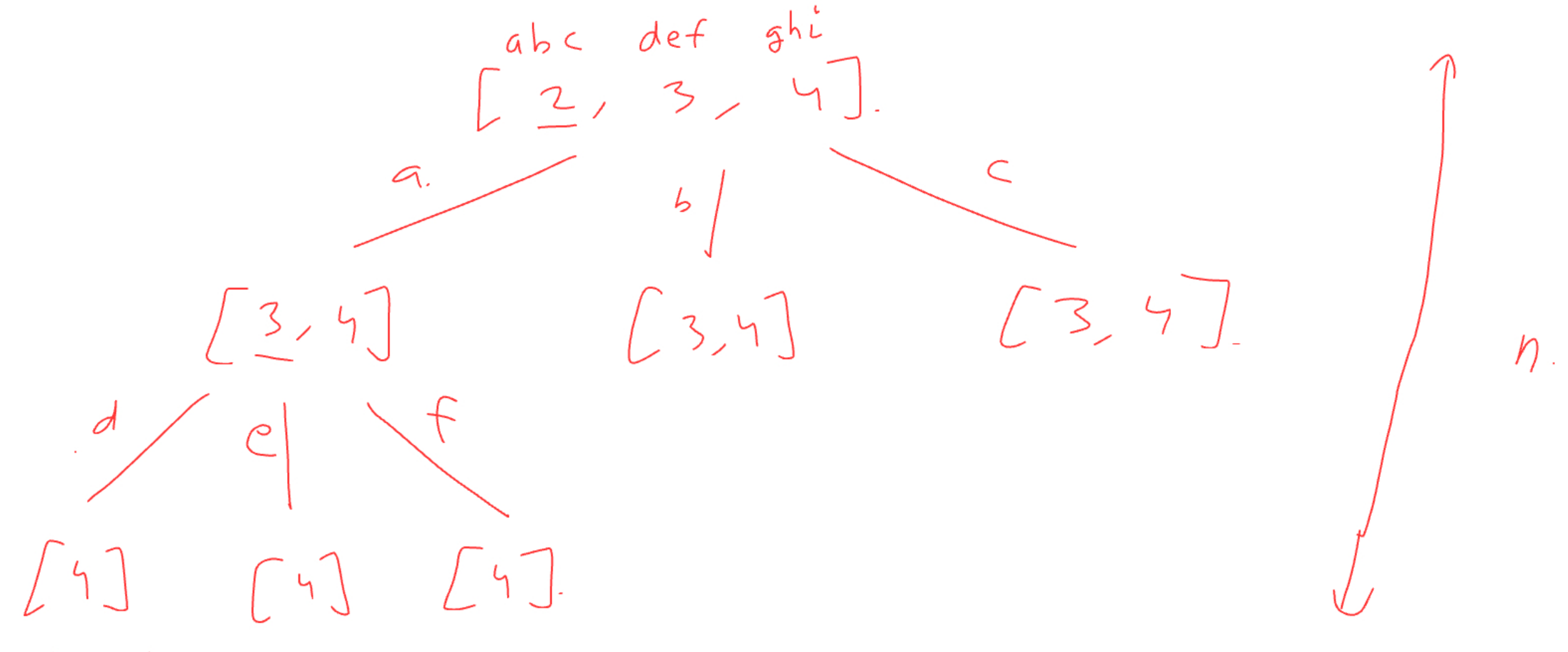

abc def ghi
[2, 3, 4]
a
b
c
[3, 4]
[3, 4]
[3, 4]
d
e
f
[4]
[4]
[4]
g h i
[] [] []
n

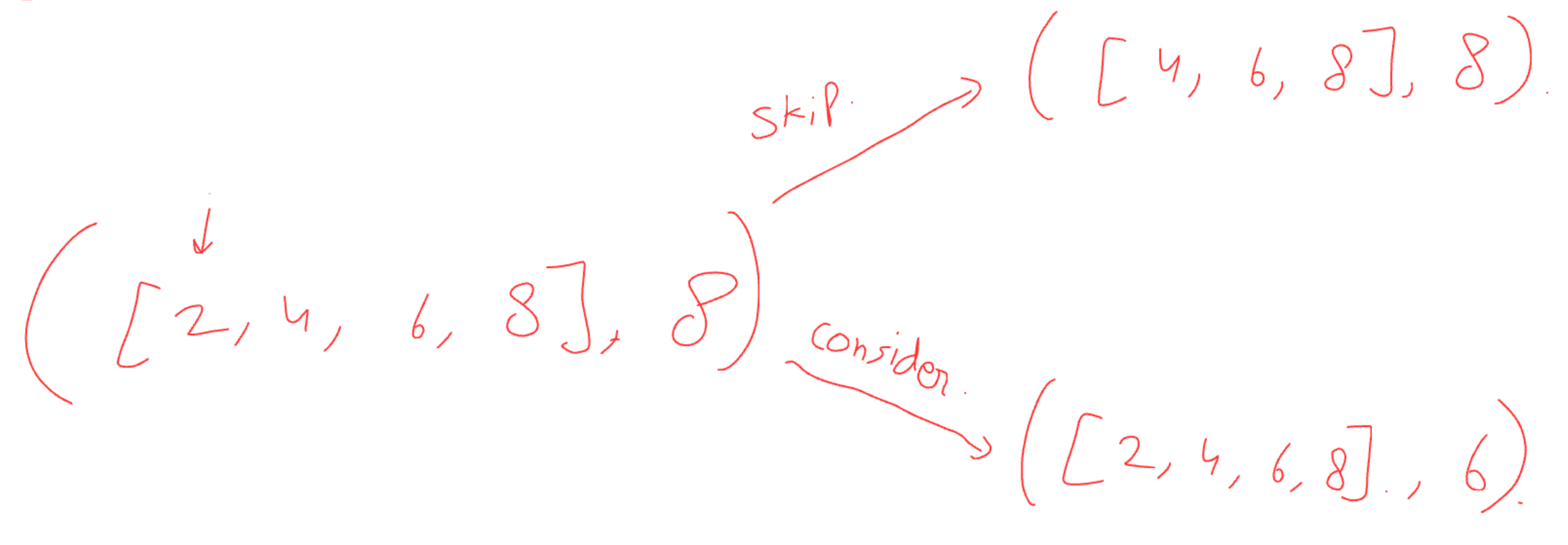
$TC = O(4^n)$

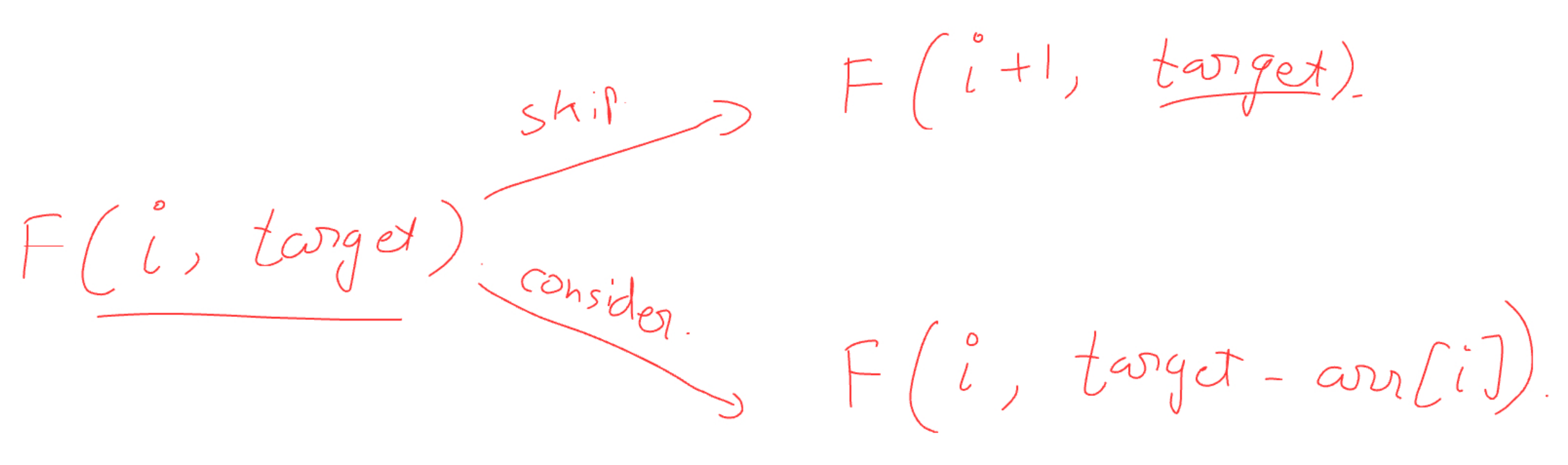
$AS = O(n+n) = O(n)$

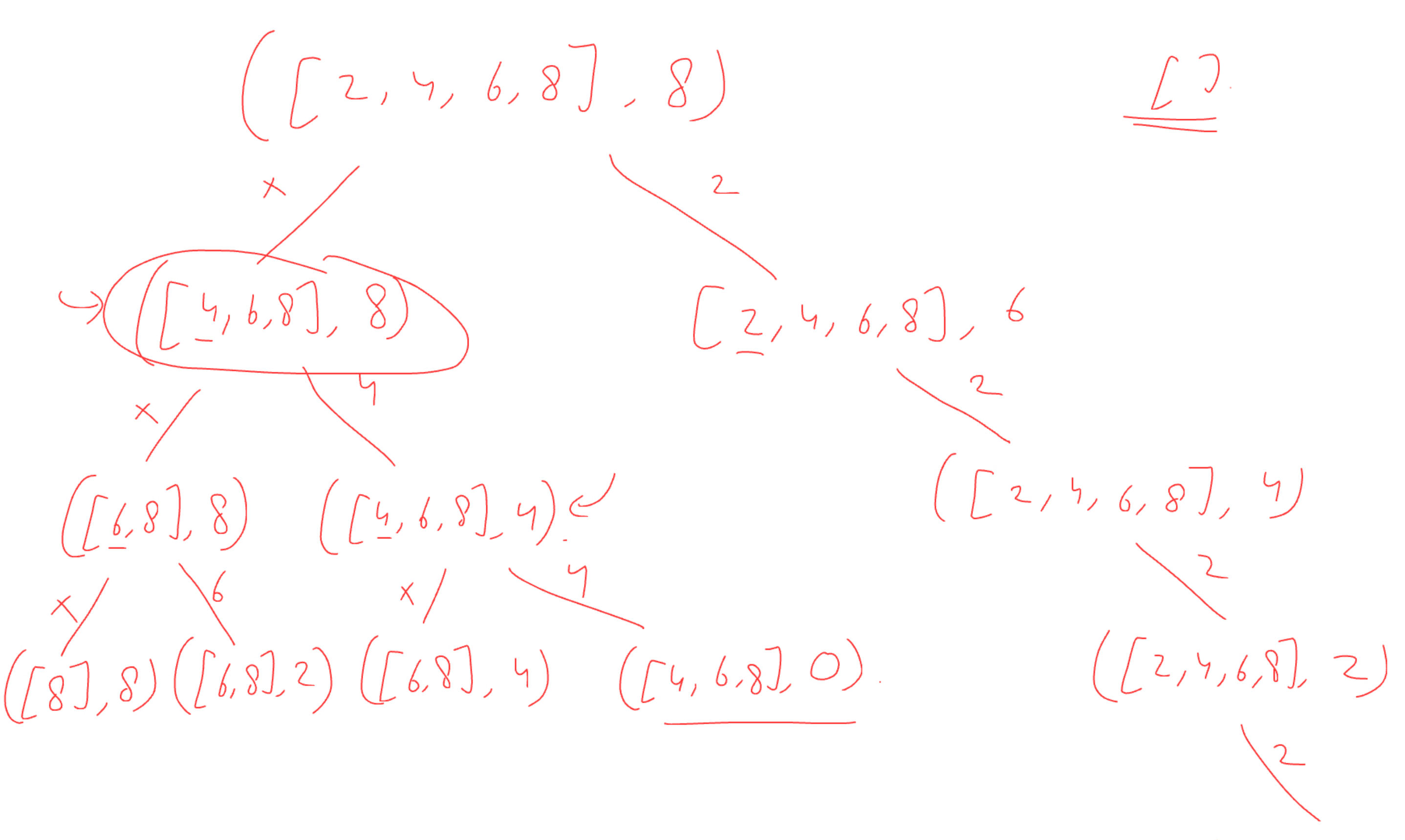
$[\overset{\downarrow}{2}, 4, 6, 8]$ target = 8.

[2, 2, 2, 2]. [2, 2, 4] [2, 6]

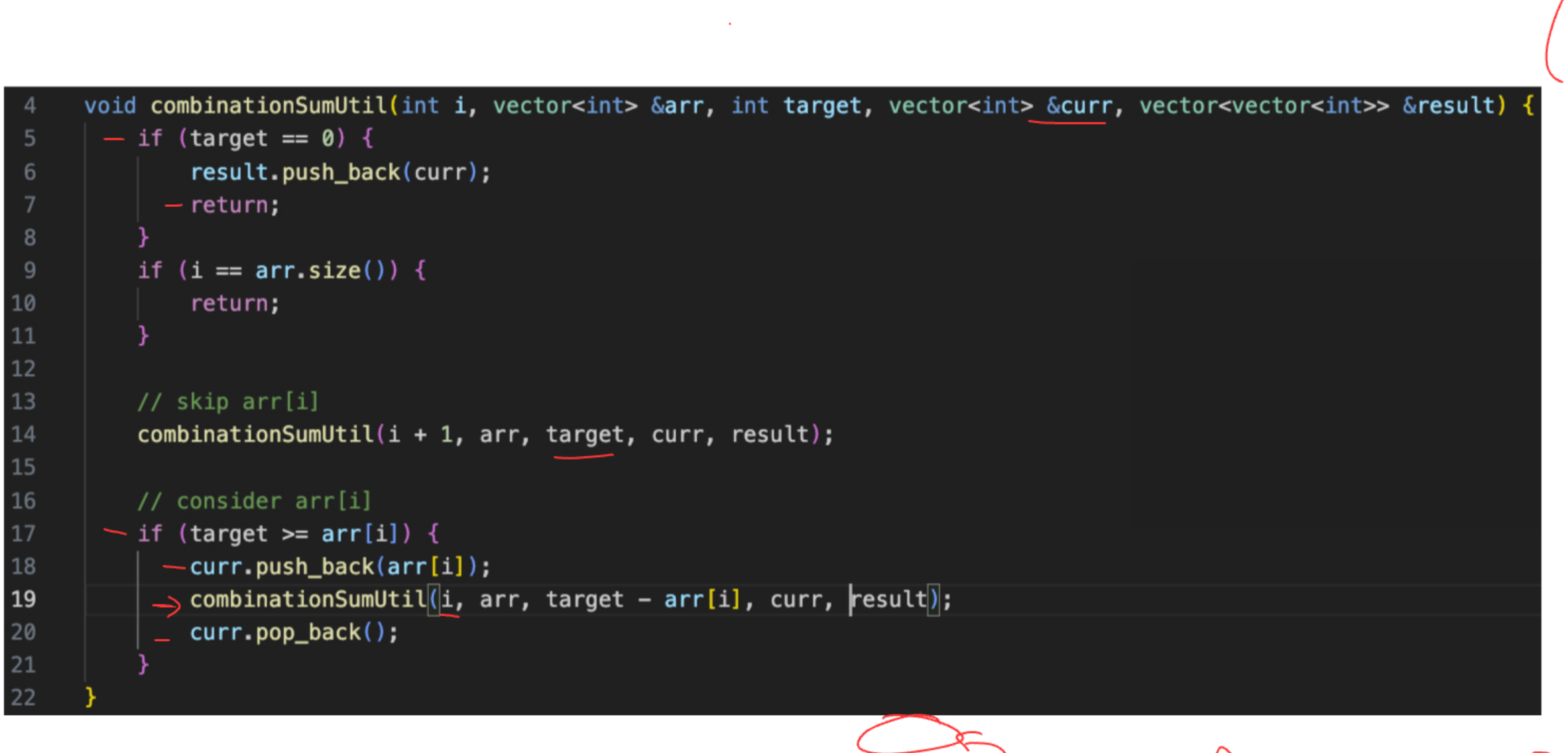
[4, 4] [8].


([2, 4, 6, 8], 8)
skip
([4, 6, 8], 8)
consider
([2, 4, 6, 8], 6)


F(i, target)
skip
F(i+1, target)
consider
F(i, target - arr[i])

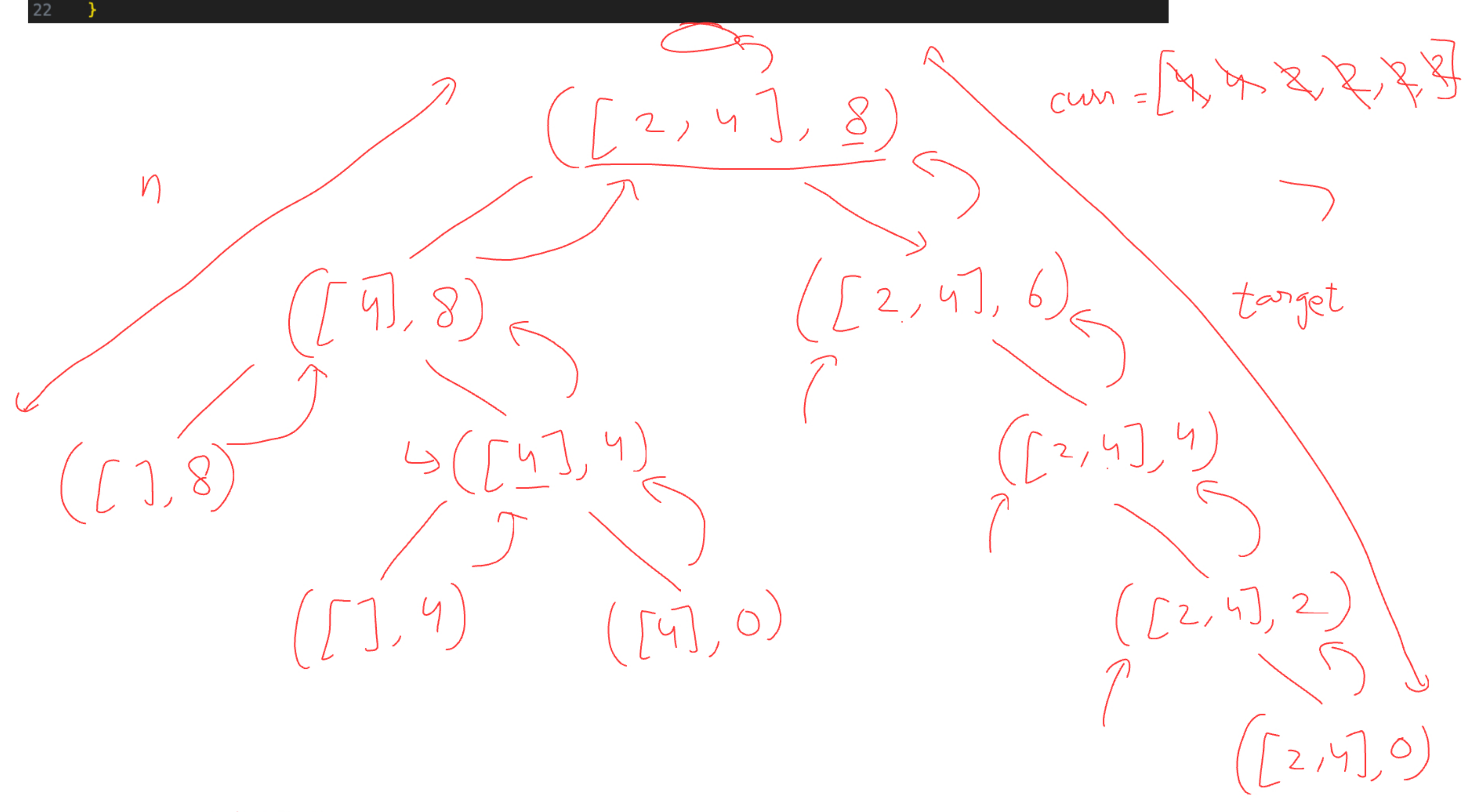

([2, 4, 6, 8], 8)
[ ]
x
2
([4, 6, 8], 8)
[2, 4, 6, 8], 6
x
4
2
([6, 8], 8)
([4, 6, 8], 4)
([2, 4, 6, 8], 4)
x
6
x
4
2
([8], 8)
([6, 8], 2)
([6, 8], 4)
([4, 6, 8], 0)
([2, 4, 6, 8], 2)
2
([2, 4, 6, 8], 0)

res:
[4, 4].
[2, 2, 2, 2].


```
void combinationSumUtil(int i, vector<int> &arr, int target, vector<int> &curr, vector<vector<int>> &result) {
    if (target == 0) {
        result.push_back(curr);
        return;
    }
    if (i == arr.size()) {
        return;
    }

    // skip arr[i]
    combinationSumUtil(i + 1, arr, target, curr, result);

    // consider arr[i]
    if (target >= arr[i]) {
        curr.push_back(arr[i]);
        combinationSumUtil(i, arr, target - arr[i], curr, result);
        curr.pop_back();
    }
}
```


n
([2, 4], 8)
curr = [~~4~~, ~~4~~, ~~2~~, ~~2~~, ~~2~~, ~~2~~]
target
([4], 8)
([2, 4], 6)
([ ], 8)
([4], 4)
([2, 4], 4)
([ ], 4)
([4], 0)
([2, 4], 2)
([2, 4], 0)

~~TC~~:

$H = \max(n, target)$

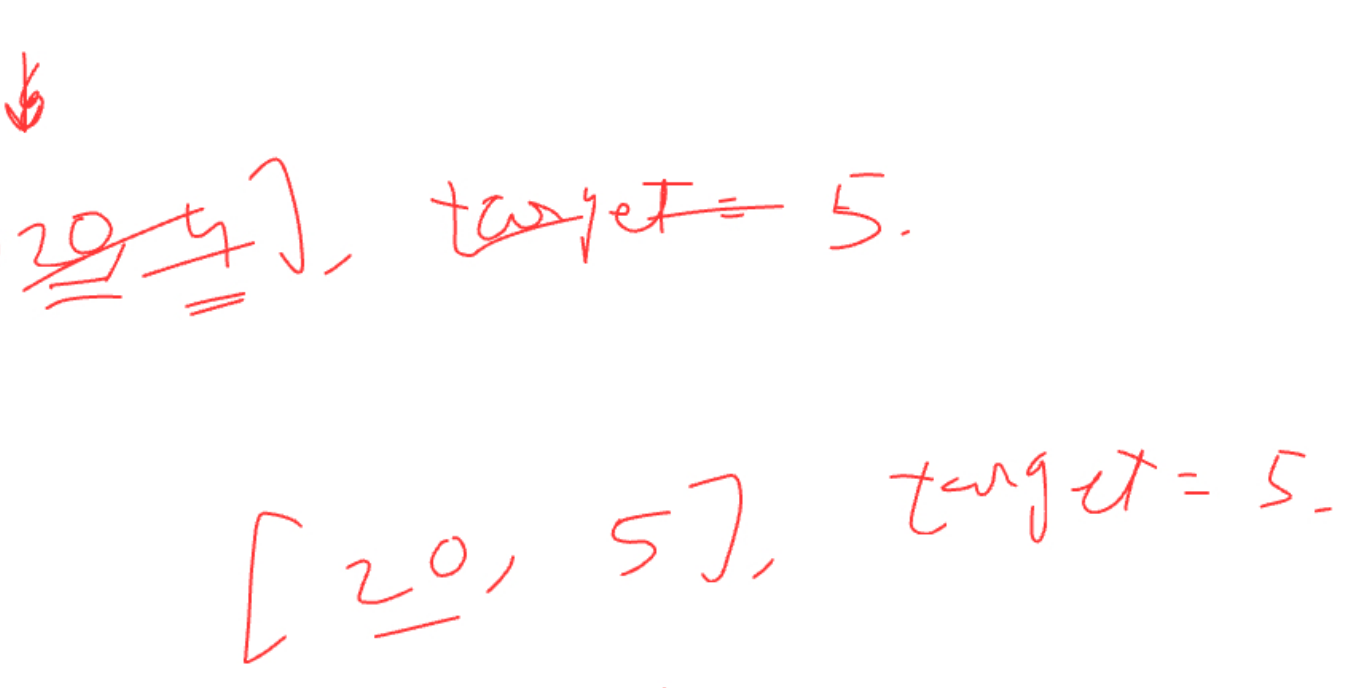
$TC = O(2^H) = O(2^{\max(n, target)})$

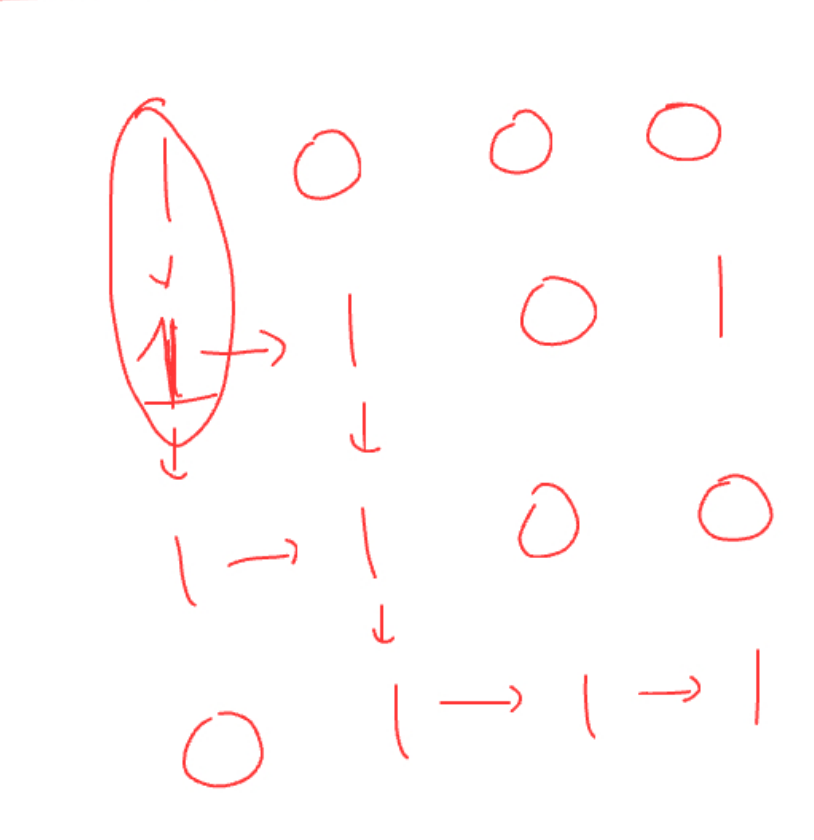
AS => Rec. stack space = $O(\max(n, target))$

curr comb = $O(target)$

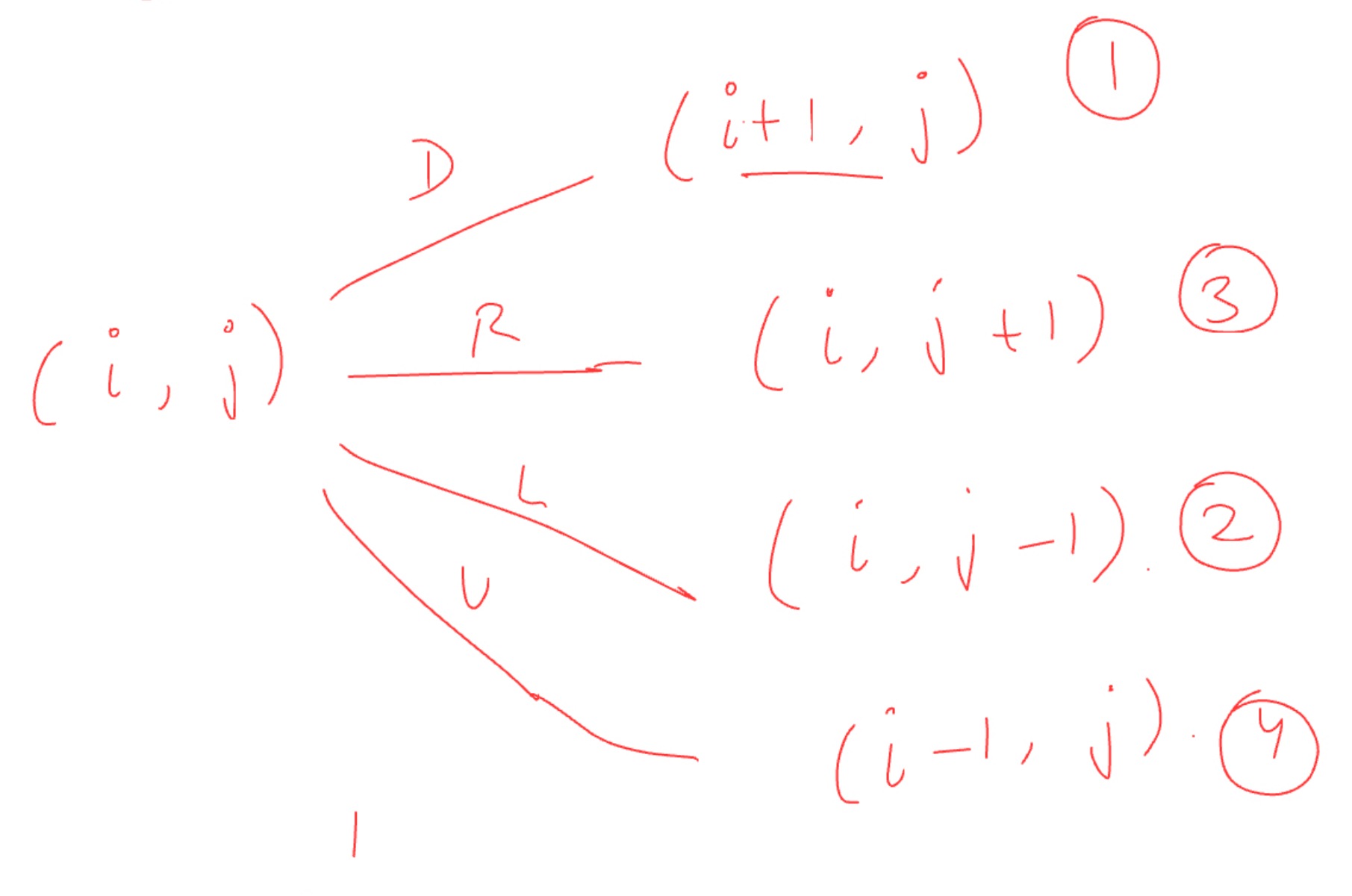
$AS = O(\max(n, target))$

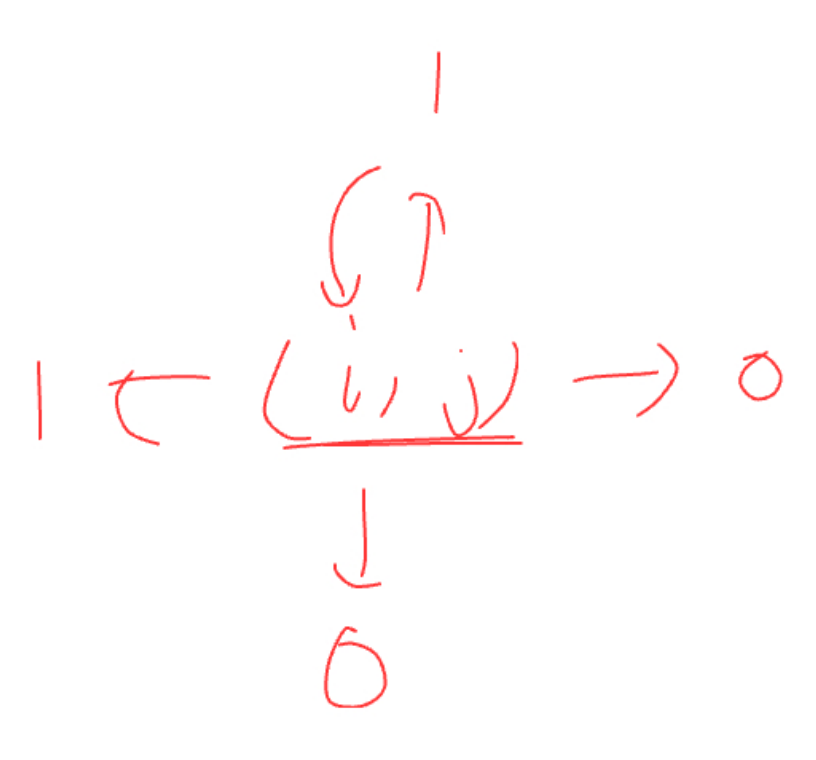
[~~20~~ ~~20~~, ~~4~~], ~~target~~ 5.

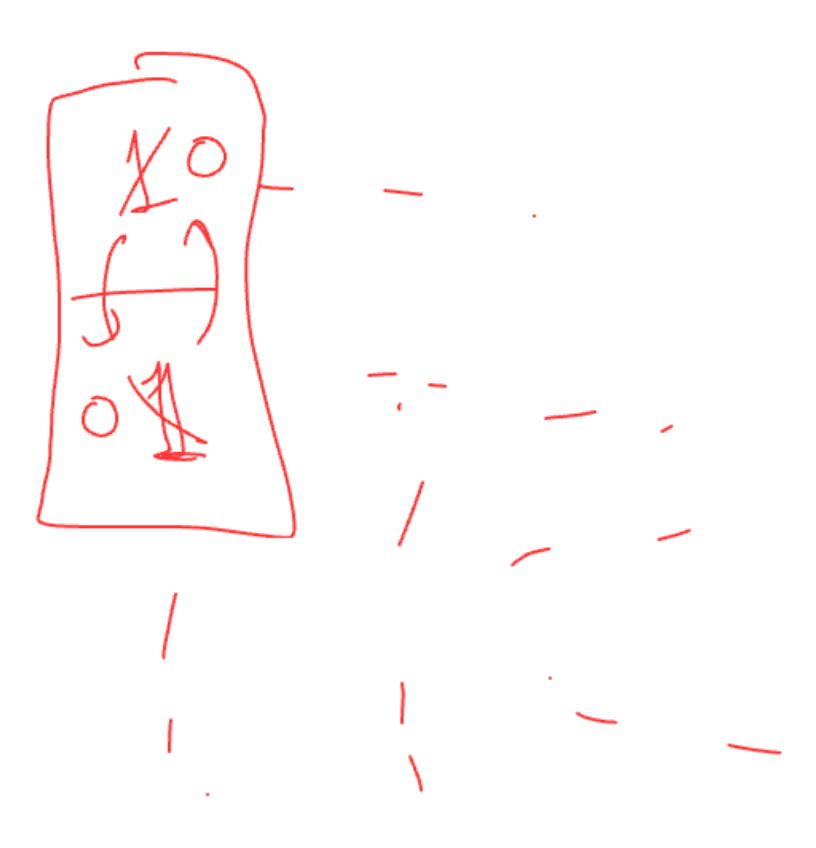

[20, 5], target = 5.
([5], 5)
([20, 5], -15)

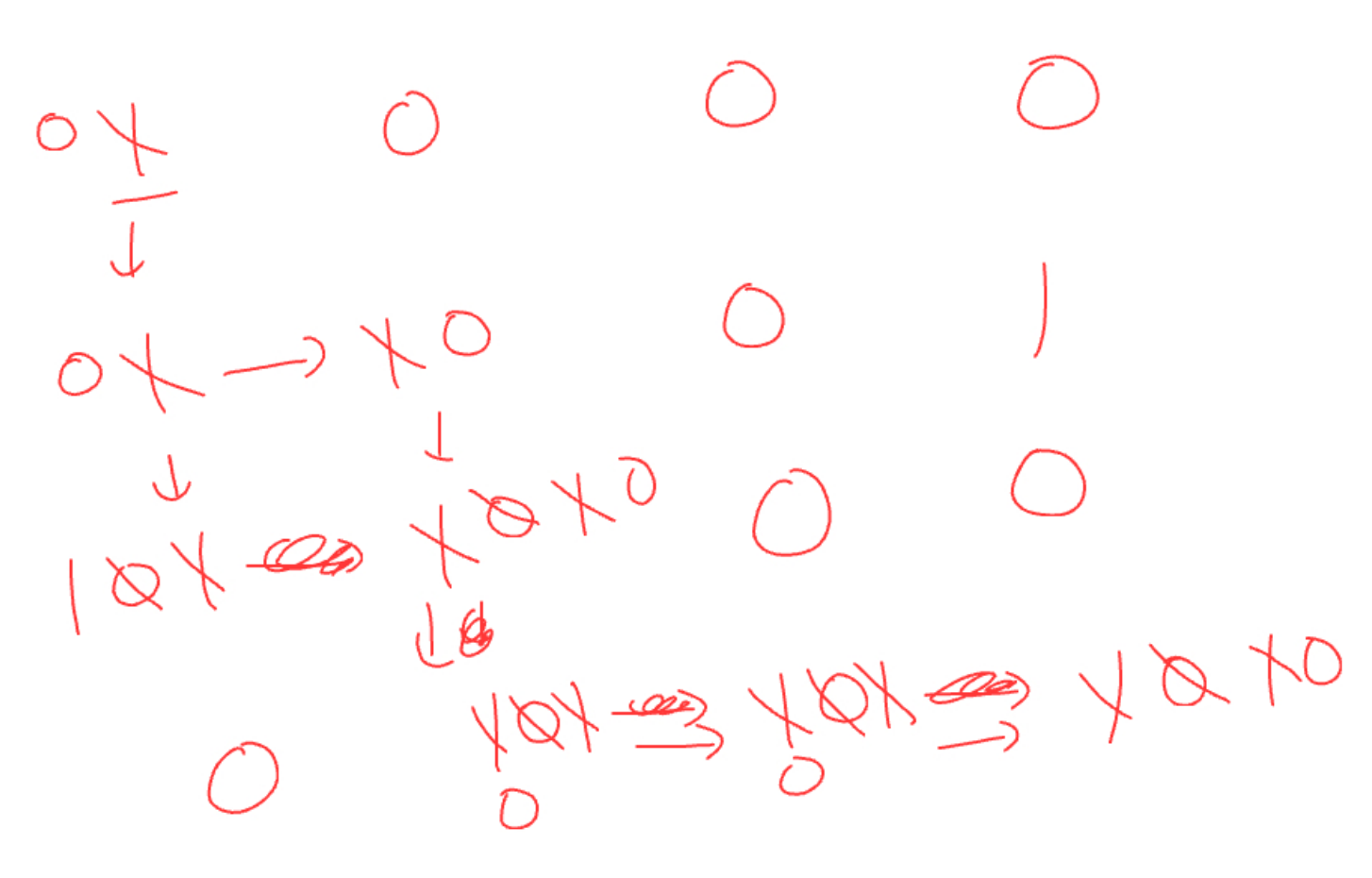


DDRDRR.
DRDDRR

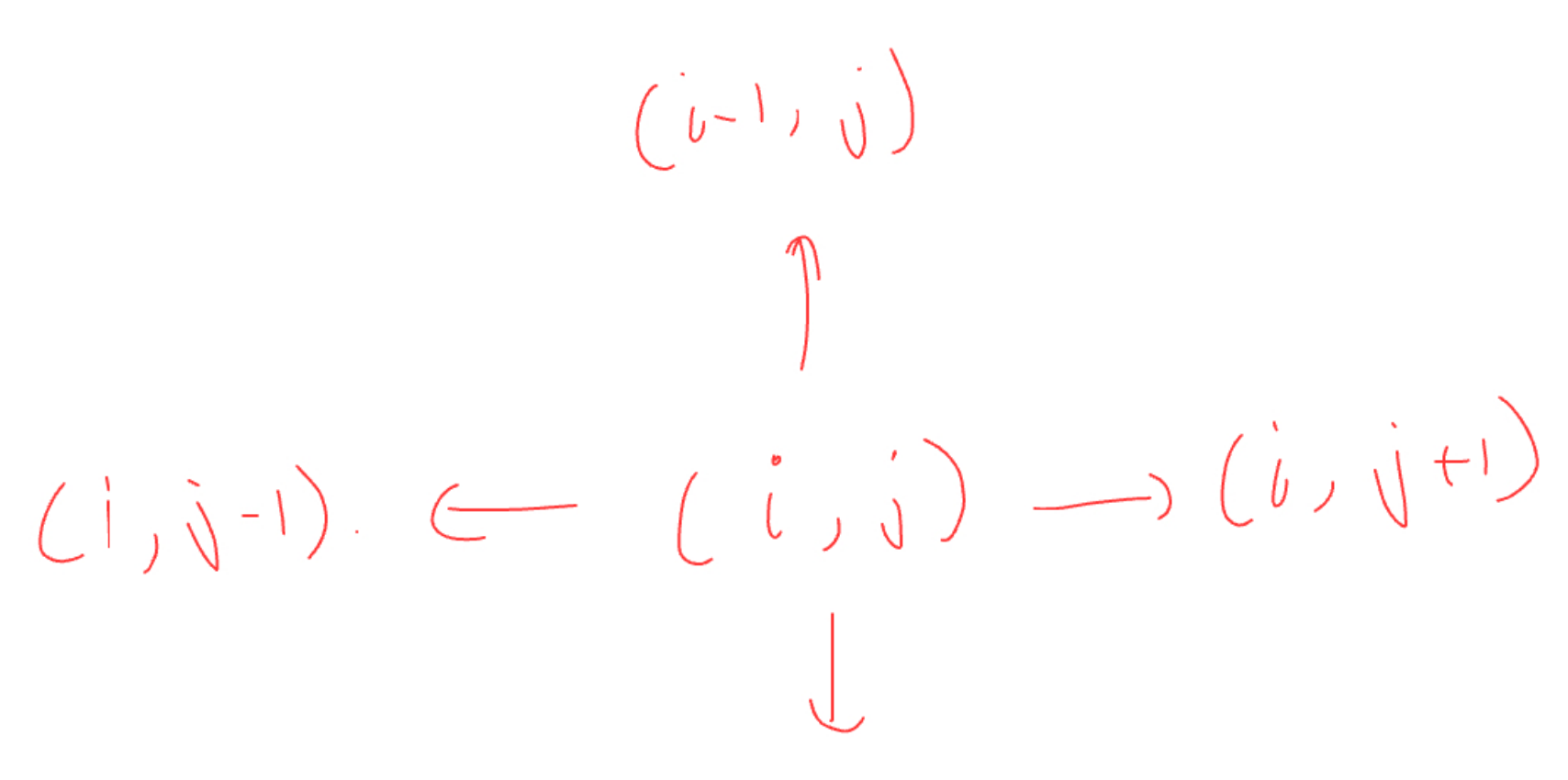

(i, j)
D
(i+1, j) ①
R
(i, j+1) ③
L
(i, j-1) ②
U
(i-1, j) ④



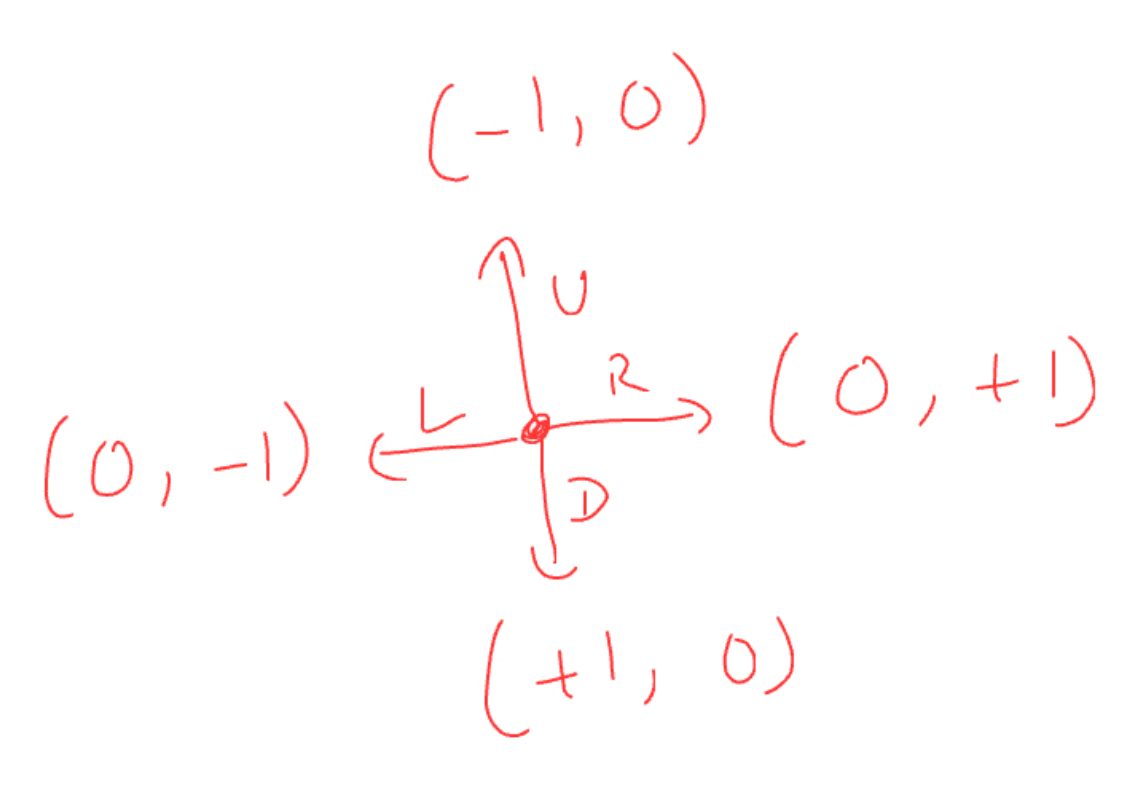


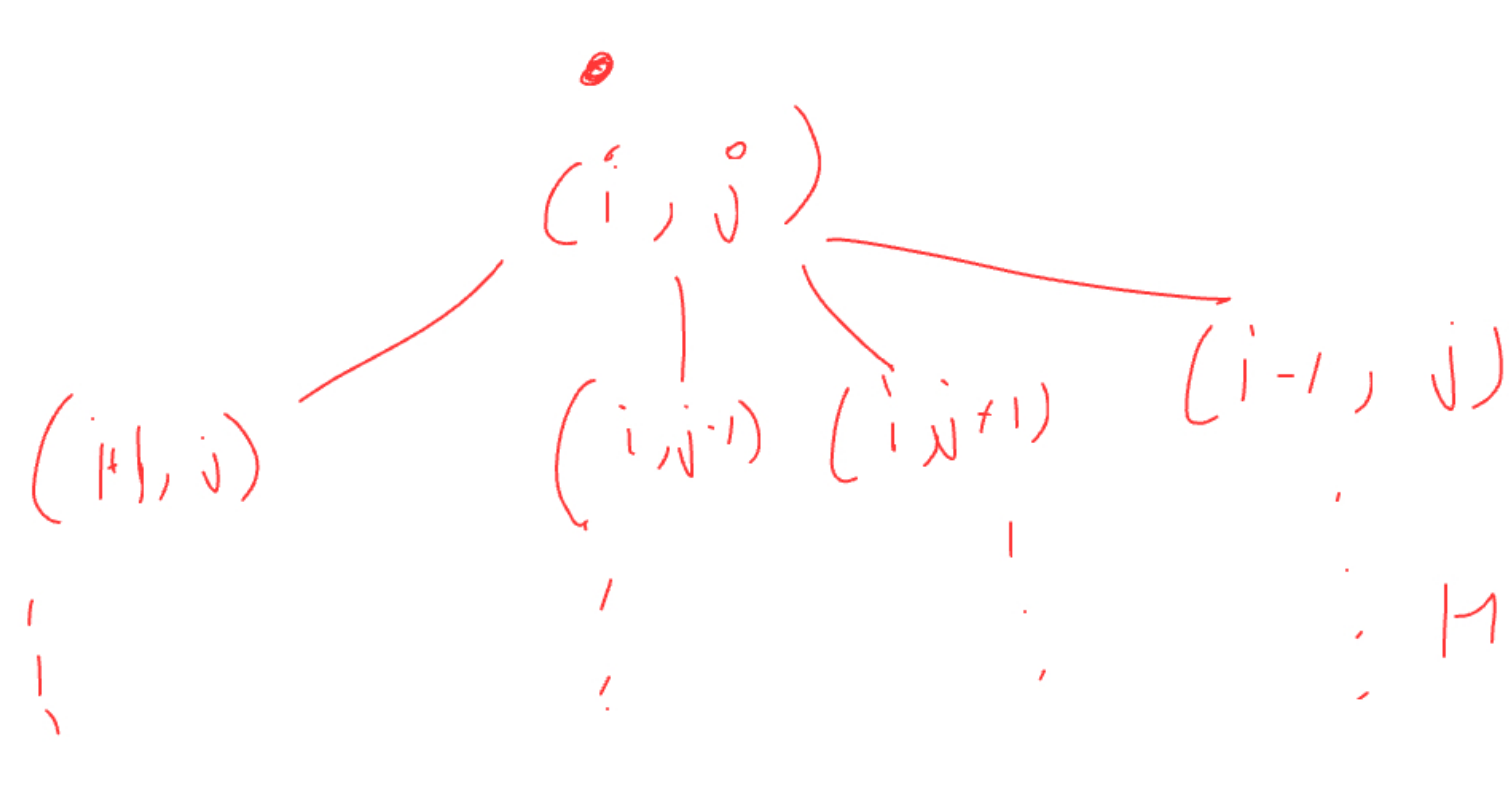


DLRU


(i-1, j)
(i, j-1)
(i, j)
(i, j+1)
(i+1, j)

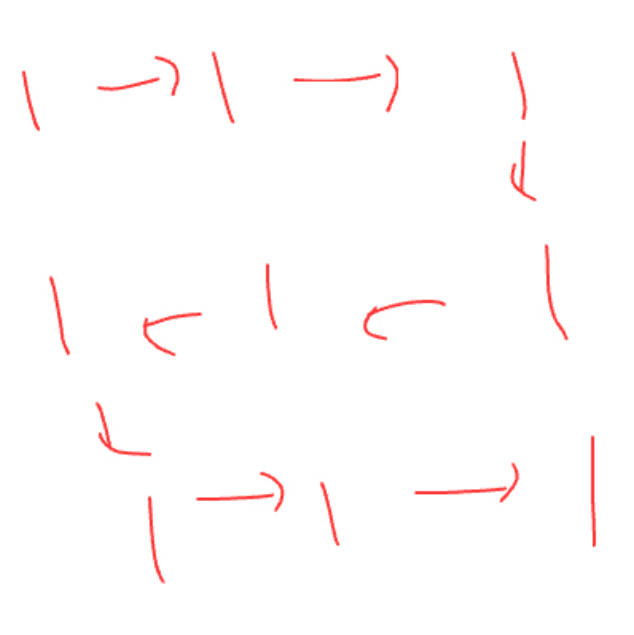
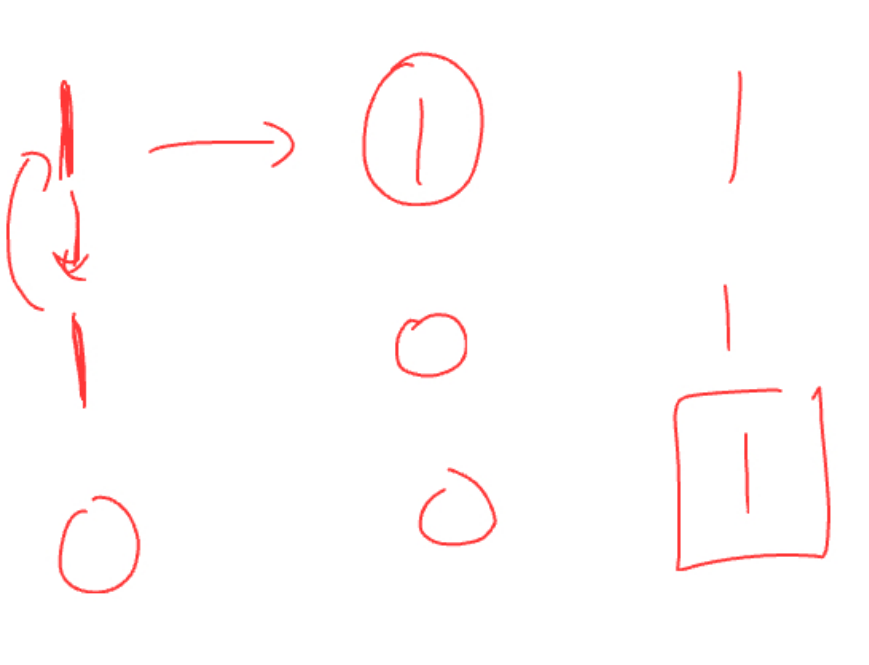
DLRU


(-1, 0)
U
(0, -1)
L
R
(0, +1)
D
(+1, 0)


(i, j)
(i+1, j)
(i, j-1)
(i, j+1)
(i-1, j)

$H = n^2$.

$TC = O(4^{n^2})$.

$AS = O(n^2)$.